

Engineering Document E04 — Frontend Architecture Blueprint

Product Name: HQ

Engineering Package: Version 1.0

Purpose

This document defines the frontend architecture for HQ.

The frontend is responsible for delivering a modern, fast, responsive, and intuitive user experience while communicating efficiently with the backend and AI orchestration engine.

HQ adopts a **feature-based architecture** using **Next.js App Router**, **TypeScript**, **Tailwind CSS**, **shadcn/ui**, and **TanStack Query**.

Engineering Principles

The frontend must be:

- Component-driven
- Feature-based
- Type-safe
- Accessible
- Responsive
- Performant
- Reusable
- Maintainable

Business logic should remain outside UI components whenever possible.

Technology Stack

Framework

- Next.js (App Router)

Language

- TypeScript

Styling

- Tailwind CSS

Component Library

- shadcn/ui

Icons

- Lucide React

Animations

- Framer Motion

Forms

- React Hook Form

Validation

- Zod

Data Fetching

- TanStack Query

State Management

- Zustand

Theme

- next-themes

Charts

- Recharts

Frontend Architecture

User

↓

Next.js App Router

↓

Layouts

↓

Pages

↓

Feature Modules

↓

Components

↓

Hooks

↓

API Client

↓

NestJS Backend

Every layer has a single responsibility.

Folder Structure

```
apps/web/  
src/  
app/  
features/  
components/  
hooks/  
services/  
stores/  
lib/  
providers/  
styles/  
types/  
config/  
constants/
```

The application is organized around business features instead of pages.

Feature-Based Organization

Each feature owns its own logic.

Example:

```
features/  
  authentication/  
  dashboard/  
  missions/
```

executives/
organizations/
billing/
analytics/
settings/
notifications/
assets/

Each feature contains:

- Components
- Hooks
- API
- Types
- Validation
- State
- Utilities

Features remain independent.

App Router Structure

app/
(auth) /
(dashboard) /
(marketing) /
api/
layout.tsx
page.tsx
loading.tsx
error.tsx
not-found.tsx

Route Groups separate application areas without affecting URLs.

Layout Hierarchy

Root Layout



Authentication Layout



Dashboard Layout



Feature Layout



Page

Shared layouts minimize duplication.

Protected Routes

Authenticated users access:

- Dashboard
- Executive Board
- Missions
- Billing
- Analytics
- Settings

Unauthenticated users are redirected to authentication.

Public Routes

Public pages include:

- Landing Page
- Pricing
- About
- Contact
- Login
- Register
- Privacy Policy
- Terms of Service

Component Architecture

Three component levels:

UI Components

Reusable primitives.

Examples:

- Button
- Input
- Card
- Badge
- Dialog
- Table

Shared Components

Reusable business components.

Examples:

- Sidebar
 - Navbar
 - Executive Card
 - Mission Card
 - Dashboard Widget
-

Feature Components

Specific to a feature.

Examples:

Mission Timeline

Executive Collaboration Panel

Billing History

Asset Gallery

State Management

Use Zustand only for global UI state.

Examples:

- Theme
- Sidebar
- Notifications
- User Preferences

Avoid storing server data in Zustand.

Server State

TanStack Query manages:

- API requests
- Caching
- Background refetching
- Optimistic updates
- Pagination

Server state remains outside local stores.

API Layer

Every feature owns its API client.

Example:

```
missions/  
api/  
createMission.ts  
updateMission.ts  
deleteMission.ts  
listMissions.ts
```

Components never call `fetch()` directly.

Forms

Every form uses:

- React Hook Form
- Zod Validation

Forms include:

- Client validation
- Loading state

- Error handling
 - Success feedback
-

Design System

All UI follows the HQ Design System.

Includes:

- Colors
- Typography
- Icons
- Spacing
- Motion
- Shadows
- Border Radius

Consistency is mandatory.

Theme Support

Support:

- Light Mode
- Dark Mode
- System Theme

Theme switching should be immediate.

Animation Principles

Animations should communicate state.

Examples:

- Page transitions
- Loading indicators
- Mission progress
- Executive activity
- Notifications

Animations should never delay productivity.

Dashboard Architecture

Dashboard contains:

Overview



Mission Center



Executive Board



Analytics



Recent Activity



Quick Actions

Widgets are modular and independently maintainable.

Error Handling

Display:

- Friendly error messages
- Retry actions
- Offline indicators
- Loading states

Users should never encounter blank screens.

Loading Strategy

Use:

- Skeleton loaders
- Suspense
- Progressive loading

Avoid unnecessary blocking.

Accessibility

Support:

- Keyboard navigation
- Screen readers
- Focus management
- ARIA attributes
- Color contrast
- Reduced motion

Accessibility is a production requirement.

Responsive Design

Primary breakpoints:

- Mobile

- Tablet
- Laptop
- Desktop
- Ultra-wide

Every feature must remain usable across screen sizes.

Performance Optimization

Implement:

- Code splitting
- Lazy loading
- Dynamic imports
- Image optimization
- Font optimization
- Route prefetching

Performance should remain excellent as the application grows.

Security

Frontend responsibilities include:

- Secure token handling
- Input sanitization
- CSRF protection where applicable
- Secure file uploads
- Session validation

Sensitive logic remains on the backend.

Testing

Frontend testing includes:

- Component tests
- Hook tests
- Integration tests
- End-to-end tests
- Accessibility testing

Critical user flows require automated coverage.

Future Expansion

The architecture supports:

- Mobile application
- Desktop application
- Multi-language support
- Offline capabilities
- Team collaboration
- Real-time updates

These additions should require minimal architectural changes.

Success Criteria

The Frontend Architecture succeeds when:

- Features remain independent.
 - Components are reusable.
 - User experience is consistent.
 - Performance remains high.
 - Accessibility standards are met.
 - The codebase scales without becoming difficult to maintain.
-

Conclusion

The HQ Frontend Architecture establishes a scalable, feature-based foundation for the platform.

By combining Next.js App Router, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, Zustand, and a modular feature-oriented structure, HQ delivers a frontend capable of supporting both a polished hackathon MVP and long-term enterprise growth.